

CHEAT SHEETS

A Complete Reference for Modern YAML

- Syntax
- Scalars
- Sequences
- Mappings
- Multiline Strings
- Anchors & Aliases
- Schema & Tags
- Real-World Patterns
- Tooling

8 REFERENCE SHEETS

CONTENTS

CORE LANGUAGE	ADVANCED & APPLIED
01 Basics & Syntax Rules	06 Schema, Tags & Directives
02 Scalars & Data Types	07 Real-World YAML Patterns
03 Collections — Sequences & Mappings	08 Tooling, Validation & Language Integration
04 String Styles & Multiline	
05 Anchors, Aliases & Merges	

Sheet 1

Basics & Syntax Rules

syntax · indentation · document markers · comments · whitespace

Sheet 2

Scalars & Data Types

string · integer · float · boolean · null · timestamp · !! tags

Sheet 3

Collections — Sequences & Mappings

sequences · mappings · nesting · block style · flow style

Sheet 4

String Styles & Multiline

plain · quoted · literal block | · folded block > · chomping · escape sequences

Sheet 5

Aliases, Anchors & Merges

& anchor · * alias · << merge key · DRY config · override · multiple merges

Sheet 6

Schema, Tags & Directives

!! tags · %YAML · %TAG · core schema · custom tags · multi-document · Norway problem

Sheet 7

Real-World YAML Patterns

GitHub Actions · Docker Compose · Kubernetes · Ansible · OpenAPI · env config

Sheet 8

Tooling, Validation & Language Integration

yamllint · PyYAML · js-yaml · SnakeYAML · YAML↔JSON · validation · editors

WHAT IS YAML?

YAML (YAML Ain't Markup Language) is a human-readable data serialization format commonly used for **configuration files**, CI/CD pipelines, Infrastructure-as-Code, and data exchange between systems. YAML files use the extensions `.yaml` or `.yml` and are parsed by a YAML processor into native data structures (maps, lists, scalars). Unlike JSON, YAML is whitespace-sensitive — indentation defines structure, and **tabs are never allowed** for indentation.

COMMON USE CASES

TOOL / PLATFORM	YAML ROLE
GitHub Actions	Workflow definition (.github/workflows/*.yml)
Docker Compose	Service, network, and volume config
Kubernetes	Resource manifests (Pod, Deployment, Service)
Ansible	Playbooks, inventory, variable files
OpenAPI / Swagger	API schema and endpoint definitions
Helm	Chart values and template overrides
Spring Boot	application.yml configuration
ESLint / Prettier	.eslintrc.yml, .prettierrc.yml

CORE RULES

- › Indentation uses **spaces only** — tabs cause parse errors
- › Consistent indent depth required within a block (2 or 4 spaces typical)
- › **#** starts a comment — extends to end of line
- › Strings do not require quotes unless they contain special characters
- › **---** marks the start of a document (optional for single-doc files)
- › **...** marks the end of a document (optional)
- › YAML is a superset of JSON — all valid JSON is valid YAML
- › Keys and values are separated by **:** (colon + space)

DOCUMENT MARKERS & SPECIAL SYNTAX

SYNTAX	MEANING
---	Start of document / separator between docs
...	Explicit end of document
#	Comment (inline or full line)
:	Key-value separator (colon + space required)
-	Sequence item (dash + space)
 	Literal block scalar (preserves newlines)
>	Folded block scalar (folds newlines)
& / *	Anchor definition / alias reference
!!	Explicit type tag (!!str, !!int, etc.)
{ }	Flow mapping (inline map)
[]	Flow sequence (inline list)

ANATOMY OF A YAML FILE

```
%YAML 1.2           # optional version directive
---                # document start marker

# Application configuration
app:
  name: my-service   # string value (unquoted)
  version: 2.1.0     # string (looks like semver)
  port: 8080         # integer
  debug: false       # boolean

server:
  host: "localhost"  # quoted string
  timeout: 30        # integer (seconds)

features:           # mapping with list value
- authentication
- rate-limiting
- logging

database:
  url: postgres://db:5432 # colon in value — no quotes needed here
  pool_size: 10

# Inline (flow) style — equivalent to block above
tags: {env: prod, region: us-east}
ports: [80, 443, 8080]

...                # optional document end marker
```

NOTES

- **%YAML 1.2** — optional directive; most parsers default to 1.1 or 1.2
- **---** — required when using directives or multiple documents; otherwise optional
- **app:** — a mapping key; child keys indented beneath it form a nested map
- **false** — unquoted booleans are auto-detected; quote to force string: **"false"**
- **features:** — a mapping whose value is a sequence (list of strings)
- **{ }** / **[]** — flow style keeps items on one line; valid anywhere block style is
- Colons inside values don't need quoting unless they appear at the start or are followed by a space

COMMON MISTAKES

- › Using **Tab** for indentation — YAML forbids tabs entirely; always use spaces
- › Forgetting the space after **:** — **key:value** is a string, not a mapping
- › Inconsistent indentation depth within a block causes parse errors
- › Writing **: #comment** directly — always leave a space: **value # comment**

TIPS

- › Use 2-space indentation — it's the community standard and most tools default to it
- › Configure your editor to show whitespace characters to catch tab vs space issues early
- › Run **yamllint** on files before committing — catches indentation and syntax errors instantly
- › Use **---** at the top of every file as a habit — it makes multi-doc scenarios trivial to add later

SUMMARY

- › **spaces only** — tabs are illegal for indentation
- › **:** — colon + space separates every key from its value
- › **#** — comments extend to end of line
- › **---** / **...** — document start and end markers
- › Block style uses indentation; flow style uses **{ }** and **[]**
- › YAML is a superset of JSON

SCALAR TYPES

A **scalar** is any single-value node in YAML — a string, number, boolean, or null. YAML performs **implicit type detection**: the parser infers the type from the value's form unless you override it with an explicit tag. This convenience is also a source of bugs — values like `yes`, `on`, `1.0`, and `2024-01-01` are automatically coerced to boolean, float, and timestamp respectively. Use **explicit tags** (`!!str`, `!!int`) or quotes to force a specific type.

TYPE DETECTION RULES

- Quoted values (`'...'` or `"..."`) are always strings
- `true`, `false` → boolean (YAML 1.2); also `yes` / `no`, `on` / `off` in YAML 1.1
- `null`, `~`, or empty value → null type
- Bare integers and floats are numeric unless quoted
- `YYYY-MM-DD` format → timestamp (date)
- `0x1F` → hex integer; `0o17` → octal
- `.inf`, `-.inf`, `.nan` → special float values

NOTES

- `'` — single-quote escape in single-quoted strings (double the quote)
- `\n` `\t` `\uXXXX` — escape sequences only work inside double-quoted strings
- `!!str 1984` — explicit tag overrides auto-detection; value becomes the string `"1984"`
- `0xFF` / `0o17` — hex and octal integer literals; not all parsers support these
- `.inf` / `.nan` — case-insensitive; also `.Inf`, `.NaN`, `.INF`
- `true` / `false` only — YAML 1.2 dropped yes/no/on/off as booleans; YAML 1.1 parsers still accept them
- Empty value after `:` is null — not an empty string; use `""` for an empty string

SCALAR EXAMPLES & TYPE COERCION

```
# — Strings —
plain: hello world           # string (no quotes needed)
single: 'it's alive'         # single quotes: ' ' escapes a quote
double: "tab:\there\nnewline" # double quotes: escape sequences active
forced_str: !!str 1984       # force "1984" not integer 1984

# — Numbers —
count: 100
hex_val: 0xFF                # 255
octal_val: 0o17              # 15
pi: 3.14159
scientific: 6.022e23
infinity: .inf
not_a_number: .nan

# — Booleans (YAML 1.2 — only true/false) —
enabled: true
disabled: false
as_string: "true"            # quoted → stays a string

# — Null —
nothing: null
tilde: ~
implicit:                    # empty value → null

# — Timestamps —
date: 2024-06-01
datetime: 2024-06-01T14:30:00Z
datetime_tz: 2024-06-01T14:30:00+05:30
date_str: !!str 2024-06-01  # force string, not date
```

SCALAR TYPES & EXAMPLES

TYPE	EXAMPLE YAML	PARSED AS
String	hello world	"hello world"
String	'can't stop'	"can't stop"
String	"line\nbreak"	line + newline + break
Integer	42	42 (int)
Integer	0xFF	255 (hex)
Integer	0o77	63 (octal)
Float	3.14	3.14 (float)
Float	1.5e3	1500.0
Float	.inf	Infinity
Float	.nan	NaN
Boolean	true / false	true / false
Null	null / ~ / (empty)	null
Timestamp	2024-06-01	date object
Timestamp	2024-06-01T12:00:00Z	datetime (UTC)

EXPLICIT TYPE TAGS (!!)

TAG	FORCES TYPE	EXAMPLE
!!str	String	!!str 42 → "42"
!!int	Integer	!!int "10" → 10
!!float	Float	!!float 1 → 1.0
!!bool	Boolean	!!bool "yes" → true
!!null	Null	!!null "" → null
!!seq	Sequence	forces list type
!!map	Mapping	forces map type
!!binary	Base64 blob	binary data
!!timestamp	Date/time	datetime object
!!python/...	Python type	PyYAML extension

COMMON MISTAKES

- `yes` / `no` become booleans in YAML 1.1 parsers — quote them if you mean the string `"yes"`
- Version strings like `1.10` parse as float `1.1` — always quote version numbers
- Empty value is `null`, not `""` — use explicit `""` when an empty string is required
- Timestamps are auto-parsed — quote ISO dates when you need strings (e.g. filenames)

TIPS

- When in doubt, quote it — quoted scalars are always strings regardless of content
- Use `!!str` tags at the key level for self-documenting intent when type is non-obvious
- Know your parser's YAML version — PyYAML defaults to 1.1; `ruamel.yaml` and most modern parsers support 1.2
- Single quotes are safer than double quotes when values contain backslashes (no escape processing)

SUMMARY

- implicit typing** — parser infers type from value shape
- `!!str` / `!!int` — explicit tags override inference
- `true` / `false` — only booleans in YAML 1.2
- `null` / `~` / empty — all represent null
- `YYYY-MM-DD` — auto-parsed as timestamp; quote to prevent

THE TWO COLLECTION TYPES

YAML has two collection types: **sequences** (ordered lists) and **mappings** (key-value dictionaries). Both can be written in **block style** (multi-line, indented) or **flow style** (inline, using `[]` and `{}`). Collections nest freely — a mapping value can be a sequence, a sequence item can be a mapping, and any depth of nesting is valid. The indentation level of a child must be *greater than* its parent's key.

COLLECTION RULES

- › Sequence items start with `-` (dash + space)
- › Mapping entries use `key: value` (colon + space)
- › Flow sequences: `[a, b, c]` — comma-separated in brackets
- › Flow mappings: `{key: val, key2: val2}`
- › Keys must be unique within the same mapping level
- › Keys can be any scalar (string, int, bool) — strings most common
- › Complex keys (multi-line) use the `?` indicator
- › Empty sequence: `[]` · Empty mapping: `{}`

NOTES

- `- item` — the dash is part of the indentation; the value starts after the space
- `server:` then indented children — the mapping's value is itself a mapping
- `users: + - name:` — sequence of mappings; each `-` begins a new object
- The `-` indicator counts as 2 spaces of indentation for its children
- `environments: production: servers:` — three levels of nested mappings
- `[[1, 0], [0, 1]]` — flow sequences can nest; useful for matrices or coordinate pairs
- Block and flow styles can be mixed freely — a block mapping can have a flow sequence as a value

NESTING COLLECTIONS

```
# — Simple sequence —————
fruits:
  - apple
  - banana
  - cherry

# — Flow sequence (equivalent) —————
fruits: [apple, banana, cherry]

# — Mapping with nested mapping —————
server:
  host: localhost
  port: 8080
  tls:
    enabled: true
    cert: /etc/ssl/cert.pem

# — Sequence of mappings (list of objects) —————
users:
  - name: Alice
    role: admin
    active: true
  - name: Bob
    role: viewer
    active: false

# — Mixed nesting: map → list → map —————
environments:
  production:
    servers:
      - host: prod-1.example.com
        weight: 100
      - host: prod-2.example.com
        weight: 100
  staging:
    servers:
      - host: stage.example.com
        weight: 100

# — Flow style for compact data —————
matrix: [[1, 0], [0, 1]]
rgb: {r: 255, g: 128, b: 0}
```

SEQUENCE PATTERNS

PATTERN	YAML
Block list	<code>- item1 / - item2</code>
Flow list	<code>[item1, item2, item3]</code>
Empty list	<code>key: []</code>
Nested lists	Indent — items under a <code>-</code>
List of maps	<code>- name: Alice / age: 30</code>
Inline list of maps	<code>- {name: Alice, age: 30}</code>
Single-item list	<code>- only_item</code>

MAPPING PATTERNS

PATTERN	YAML
Simple map	<code>key: value</code>
Flow map	<code>{a: 1, b: 2}</code>
Empty map	<code>key: {}</code>
Nested map	Child keys indented under parent key
Map with list value	<code>ports: then - 80 / - 443</code>
Integer key	<code>404: Not Found</code>
Complex key	<code>? [1, 2] / : value</code>

COMMON MISTAKES

- › Not indenting map children — siblings at the same level as the parent key break the structure
- › Forgetting the space after `-` in sequences — `-item` is a string, not a list item
- › Duplicate keys in the same mapping — most parsers silently use the last value; behavior is undefined
- › Mixing indentation depths within the same block — 2-space parent cannot have 3-space children

TIPS

- › Use block style for config files; reserve flow style for short, simple values that read naturally inline
- › Sequence-of-mappings is the most common YAML pattern — master `- key: value` nesting first
- › Paste YAML into an online validator (yaml-online-parser.appspot.com) to visualize the parsed tree

SUMMARY

- › `- item` — block sequence item
- › `[a, b]` — flow sequence
- › `key: value` — mapping entry
- › `{k: v}` — flow mapping
- › Collections nest to any depth; indentation defines the parent-child relationship

FIVE WAYS TO WRITE A STRING

YAML provides five string styles. **Plain** scalars need no quotes and are the most readable. **Single-quoted** strings treat all characters literally (no escape sequences) except `' '` for a literal quote. **Double-quoted** strings support backslash escapes (`\n`, `\t`, `\uXXXX`). **Literal block** (`|`) preserves newlines exactly — ideal for scripts or file contents. **Folded block** (`>`) converts single newlines to spaces and double newlines to paragraph breaks — ideal for long prose.

ESCAPE SEQUENCES (DOUBLE-QUOTED ONLY)

- › `\n` — newline
- › `\t` — tab
- › `\\` — literal backslash
- › `\"` — literal double quote
- › `\r` — carriage return
- › `\0` — null character
- › `\uXXXX` — Unicode code point (4 hex digits)
- › `\uxxxxxxxx` — Unicode code point (8 hex digits)
- › `\xXX` — 8-bit Unicode (2 hex digits)

NOTES

- `'C:\Users\...'` — single quotes are the safest choice for Windows paths; no escape processing
- `' '` — the only escape in single-quoted strings; doubles a single quote character
- `|` — literal block; content indented below the key, indentation is stripped from output
- `|-` — strip modifier removes all trailing newlines; useful when embedding in templates
- `>` — folded block; single line breaks become spaces; blank line creates `\n\n`
- `>-` — folded + strip; most common for long single-line config values
- Block scalar indentation is detected automatically from the first content line; or set explicitly with `|2`

ALL STRING STYLES COMPARED

```
# — Plain scalar —
greeting: Hello World           # → "Hello World"
version: "1.0"                 # quoted to prevent float parse

# — Single-quoted —
path: 'C:\Users\admin\file.txt' # backslash literal, no escaping
quote: 'she said 'hello''      # ' ' inside single quotes = literal '

# — Double-quoted —
newlines: "first\nsecond\nthird" # → first=second=third
unicode: "\u00A9 2024"           # → © 2024
tab: "col1\tcol2"                # → col1=col2

# — Literal block (|) — newlines preserved —
script: |
  #!/bin/bash
  echo "Starting..."
  cd /app && npm start
  # Each line kept exactly, including the newline at end

# — Literal strip (|-) — trailing newlines removed —
inline_script: |-
  echo "no trailing newline"
  exit 0

# — Folded block (>) — single newlines become spaces —
description: >
  This is a long description
  that spans multiple lines but
  will be joined into one paragraph.

  A blank line creates a new paragraph.

# — Folded strip (>-) — fold + no trailing newline —
summary: >-
  One long line with
  no trailing newline at end
```

STRING STYLE COMPARISON

STYLE	INDICATOR	ESCAPE SEQS	BEST FOR
Plain	none	✗	Simple words, most values
Single-quoted	'...'	✗	Strings with special chars, no backslash
Double-quoted	"..."	✓	Strings needing <code>\n</code> , <code>\t</code> , unicode
Literal block		✗	Scripts, file content, preserve newlines
Folded block	>	✗	Long prose, word-wrapped text

BLOCK CHOMPING INDICATORS

INDICATOR	TRAILING NEWLINES	EXAMPLE
or >	One newline kept (clip, default)	>
- or >-	All trailing newlines removed (strip)	- >-
+ or >+	All trailing newlines kept (keep)	+ >+
2 or 4	Explicit indentation (2 or 4 spaces)	2 4
2-	Indent indicator + strip combined	2-

COMMON MISTAKES

- › Using `|` when you want a single-line result — use `>-` to fold lines and strip the trailing newline
- › Expecting `\n` to work in single-quoted strings — escapes only work in double-quoted strings
- › Inconsistent indentation inside block scalars — the first content line sets the indentation baseline
- › Not accounting for the trailing newline in `|` blocks when passing to shell commands

TIPS

- › Use `|-` for embedded scripts and commands — it removes the trailing newline that would otherwise be passed to the shell
- › Use `>-` for long descriptions in CI/CD configs — keeps the file readable while producing a single string
- › Single-quoted strings are ideal for regex patterns and file paths — no backslash surprises

SUMMARY

- › **plain** — no quotes; simplest; auto-typed
- › **'...'** — single-quoted; literal backslashes; `' '` for quote
- › **"..."** — double-quoted; full escape sequences
- › **|** — literal block; preserves newlines
- › **>** — folded block; single breaks → spaces
- › **-** modifier strips trailing newlines; **+** keeps them all

REUSE WITHOUT REPETITION

YAML's **anchor** (&name) and **alias** (*name) mechanism lets you define a value once and reference it multiple times, keeping configuration DRY. An anchor marks a node; an alias inserts a copy of that node elsewhere in the document. The **merge key** (<<) extends this by merging a mapping's contents into another mapping, with the ability to override specific keys after the merge. Anchors and aliases are resolved at parse time — the consumer sees fully expanded data.

ANCHOR & ALIAS RULES

- › **&name** — defines an anchor; name can contain letters, digits, hyphens
- › ***name** — inserts a copy (alias) of the anchored node
- › Anchors must appear *before* their aliases in the document
- › Aliases produce a deep copy — modifying one does not affect the other (in most parsers)
- › **<<: *anchor** — merge key; merges anchored mapping into current mapping
- › Keys defined after a merge override the merged values
- › Multiple merges: **<<: [*a, *b]** — first list item has highest priority
- › Anchors work on scalars, sequences, and mappings

NOTES

- **&defaults** — anchor name follows immediately; the mapping below it is the anchored node
- **<<: *defaults** — merge key; all keys from the anchor are inserted into the current map
- **timeout: 60** after merge — explicit keys after **<<** override merged values
- **[*base, *monitoring]** — multiple merge; first anchor's keys take priority over later anchors on conflict
- Aliases in string values (**"https://*domain"**) do NOT expand — ***name** only works at the node level
- ***ports** as a value inserts the full sequence — each service gets its own copy of the list
- The merge key **<<** is only valid in mappings — it has no effect in sequences

ANCHORS, ALIASES & MERGE KEYS

```
# — Define a shared anchor —————
defaults: &defaults
  retries: 3
  timeout: 30
  log_level: info

# — Simple alias — exact copy —————
worker_a:
  <<: *defaults           # merges retries, timeout, log_level
  queue: high-priority    # adds a new key

worker_b:
  <<: *defaults
  queue: low-priority
  timeout: 60             # overrides the merged timeout: 30

# — Multiple merges —————
base: &base
  image: ubuntu:22.04
  restart: unless-stopped

monitoring: &monitoring
  metrics: true
  health_check: /healthz

app:
  <<: [*base, *monitoring] # both merged; base keys win on conflict
  name: my-app
  port: 8080

# — Scalar anchor —————
domain: &domain example.com
api_url: "https://&domain/api" # NOTE: aliases in strings don't expand
                                # use anchors only at node level

# — Sequence alias —————
common_ports: &ports [80, 443, 8080]
service_a:
  ports: *ports
service_b:
  ports: *ports
```

ANCHOR / ALIAS / MERGE SYNTAX REFERENCE

SYNTAX	ROLE	EXAMPLE
&name	Define anchor on a node	defaults: &defaults
*name	Insert alias (copy of anchor)	prod: *defaults
<<: *name	Merge anchor mapping into current mapping	<<: *defaults
<<: [*a, *b]	Merge multiple anchors (first wins on conflict)	<<: [*base, *extra]
key: *name	Alias as a value in a mapping	config: *shared
- *name	Alias as a sequence item	- *server_defaults

COMMON MISTAKES

- › Placing an alias before its anchor — anchors must be defined earlier in the document
- › Expecting **"https://*domain"** string interpolation — aliases don't expand inside quoted strings
- › Overriding a key before the merge — put **<<** first, then override keys below it
- › Multiple **<<** keys in the same map — only one merge key is valid; use the list form **[*a, *b]**

TIPS

- › Use a top-level **x-** prefixed key (e.g. **x-defaults:**) for anchors you don't want parsed as real data — Docker Compose ignores **x-*** keys
- › Define all anchors near the top of the file so they're easy to find and maintain
- › Merge keys work in Kubernetes Helm values — useful for environment-specific overrides

SUMMARY

- › **&name** — define a reusable anchor
- › ***name** — insert alias (copy) of anchor
- › **<<: *name** — merge anchor mapping into current map
- › Keys after merge override merged values
- › **[*a, *b]** — merge multiple anchors; first wins on conflict

SCHEMAS & TYPE RESOLUTION

A YAML **schema** defines how untagged scalars are resolved to types. The **Failsafe schema** treats everything as a string. The **JSON schema** recognizes true/false/null/numbers exactly as JSON does. The **Core schema** (YAML 1.2 default) extends JSON with additional true/false/null forms and number formats. The **YAML 1.1 schema** (still used by many parsers) additionally recognizes yes/no/on/off as booleans — the source of many cross-version bugs. Explicit **tags** (!!str, !mytype) override schema resolution entirely.

SCHEMA & DIRECTIVE RULES

- ▶ %YAML 1.2 directive declares the YAML version; must appear before ---
- ▶ %TAG ! tag:yaml.org,2002: — shorthand for built-in tag prefix
- ▶ !! is shorthand for tag:yaml.org,2002: — so !!str = full URI tag
- ▶ Custom tags use single !: !mytag value
- ▶ Tags apply to any node: scalars, sequences, and mappings
- ▶ Parsers may not support all tags — check your library's docs
- ▶ Multi-document: multiple --- separated docs in one file

NOTES

- %YAML 1.2 — directive tells the parser which schema to use; affects boolean/null/int resolution
- %TAG ! — defines a shorthand prefix for custom tag URIs (! is the default handle)
- !!int "8080" — tag on a quoted string; forces it to integer 8080 despite the quotes
- !db-config — application-defined tag; Python's PyYAML can map this to a class constructor
- !!binary | — base64-encoded binary data in a literal block; parsers decode to bytes
- --- between documents — each block is a separate parsed document; use load_all() in most parsers
- The Norway problem: country code NO was parsed as boolean false in YAML 1.1 — always quote country codes

BUILT-IN TAGS (!! SHORTHAND)

TAG	FULL URI	RESULT TYPE
!!str	tag:yaml.org,2002:str	String
!!int	tag:yaml.org,2002:int	Integer
!!float	tag:yaml.org,2002:float	Float
!!bool	tag:yaml.org,2002:bool	Boolean
!!null	tag:yaml.org,2002:null	Null
!!seq	tag:yaml.org,2002:seq	Sequence
!!map	tag:yaml.org,2002:map	Mapping
!!binary	tag:yaml.org,2002:binary	Base64 binary data
!!timestamp	tag:yaml.org,2002:timestamp	Datetime object
!!omap	tag:yaml.org,2002:omap	Ordered map

COMMON MISTAKES

- ▶ Using NO, yes, on, off as strings in YAML 1.1 parsers — they silently become booleans
- ▶ Assuming %YAML 1.2 directive changes behavior in all parsers — PyYAML ignores it and uses 1.1 rules
- ▶ Forgetting load_all() for multi-document files — load() only returns the first document
- ▶ Using 0777 for octal — YAML 1.2 treats this as integer 777; use 0o777 instead

TIPS

- ▶ Use ruamel.yaml instead of PyYAML when you need true YAML 1.2 compliance in Python
- ▶ Quote all ISO country codes, version numbers, and date-like strings to prevent type coercion surprises
- ▶ Multi-document YAML is useful for Kubernetes manifests — multiple resources in a single kubectl apply -f file

DIRECTIVES, TAGS & MULTI-DOCUMENT

```
%YAML 1.2
%TAG ! tag:example.com,2024:
---
# — Explicit built-in tags —
port:      !!int "8080"      # force integer
version:   !!str 2.0         # force string "2.0"
enabled:   !!bool "yes"     # force boolean true
data:      !!null ""        # force null

# — Custom application tag —
connection: !db-config
  host: localhost
  driver: postgres

# — Binary data tag —
thumbnail: !!binary |
R0LGODlhDAAMAIQAAP//9/X
17unp5WmZgAAAFn515eXv
Pz7Y60juDg4J+fn50Tk6enp
56SkpGRkZCQkAAACH5BAEAA
BQALAAAAAAMAAwAAAVWoCAA

# — Multi-document file —
---
# Document 1: development config
env: development
debug: true
db_url: sqlite:///dev.db

---
# Document 2: production config
env: production
debug: false
db_url: postgres://prod-db:5432/app
```

SCHEMA GOTCHAS (THE NORWAY PROBLEM & OTHERS)

VALUE	YAML 1.1	YAML 1.2
NO	boolean false	string "NO"
yes	boolean true	string "yes"
on	boolean true	string "on"
off	boolean false	string "off"
1.0	float 1.0	float 1.0
1_000	integer 1000	string "1_000"
0777	octal 511	integer 777
1:30	integer 90 (sexagesimal)	string "1:30"

SUMMARY

- ▶ %YAML 1.2 — version directive before ---
- ▶ !! — shorthand for built-in YAML tag namespace
- ▶ !mytag — custom application-defined tag
- ▶ YAML 1.1 vs 1.2 — yes/no/on/off behavior differs significantly
- ▶ --- separates multiple documents in one file

YAML IN THE WILD

YAML is the dominant configuration language for modern DevOps, cloud infrastructure, and API tooling. Each platform has its own schema and required top-level keys — but they all share the same YAML syntax rules. Understanding the **structural patterns** of each platform helps you read unfamiliar configs quickly and write new ones without referring to documentation for every field. The code block shows minimal but complete examples of the most common platforms.

MINIMAL REAL-WORLD EXAMPLES

```
# — GitHub Actions workflow —————
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test

# — Docker Compose service —————
services:
  web:
    image: nginx:alpine
    ports: ["80:80"]
    volumes: [./html:/usr/share/nginx/html:ro]
    depends_on: [api]
  api:
    build: ./api
    environment:
      DATABASE_URL: postgres://db:5432/app

# — Kubernetes Deployment (excerpt) —————
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  labels: {app: my-app}
spec:
  replicas: 3
  selector:
    matchLabels: {app: my-app}
  template:
    spec:
      containers:
        - name: app
          image: my-app:latest
          ports: [{containerPort: 8080}]

# — Ansible task —————
- name: Install nginx
  hosts: webservers
  become: true
  tasks:
    - name: Install package
      ansible.builtin.apt:
        name: nginx
        state: present
```

NOTES

- `on: [push, pull_request]` — GitHub Actions: note `on` is also a YAML 1.1 boolean — quote it if your parser is strict: `"on":`
- `uses: actions/checkout@v4` — references a reusable action from GitHub Marketplace
- `depends_on` — Docker Compose: ensures `db` starts before `api`, but does not wait for readiness
- `apiVersion: apps/v1` — Kubernetes: version determines which fields are available; always check the API docs
- `matchLabels` — Kubernetes selector must exactly match the pod template's labels or pods won't be managed
- `become: true` — Ansible: escalates to root (sudo); required for package installation
- `ansible.builtin.apt` — fully-qualified module name; preferred in modern Ansible over bare `apt:`

PLATFORM KEY FIELDS

- › **GitHub Actions:** `on`, `jobs`, `steps`, `uses`, `run`
- › **Docker Compose:** `version`, `services`, `networks`, `volumes`
- › **Kubernetes:** `apiVersion`, `kind`, `metadata`, `spec`
- › **Ansible:** `hosts`, `become`, `tasks`, `vars`, `handlers`
- › **OpenAPI 3:** `openapi`, `info`, `paths`, `components`
- › **Helm values:** plain YAML; referenced in templates with `{{ .Values.key }}`

PLATFORM STRUCTURE SUMMARY

PLATFORM	ROOT KEYS	FILE
GitHub Actions	<code>name</code> · <code>on</code> · <code>env</code> · <code>jobs</code>	<code>.github</code>
Docker Compose	<code>version</code> · <code>services</code> · <code>networks</code> · <code>volumes</code>	<code>docker</code>
Kubernetes Pod	<code>apiVersion</code> · <code>kind</code> · <code>metadata</code> · <code>spec</code>	<code>any.yaml</code>
Kubernetes Deployment	<code>same</code> + <code>spec.template.spec.containers</code>	<code>deployment</code>
Ansible Playbook	<code>hosts</code> · <code>become</code> · <code>vars</code> · <code>tasks</code> · <code>handlers</code>	<code>playbook</code>
OpenAPI 3.x	<code>openapi</code> · <code>info</code> · <code>servers</code> · <code>paths</code> · <code>components</code>	<code>openapi</code>
Helm values	user-defined; referenced via <code>{{ .Values.x }}</code>	<code>values</code>
ESLint	<code>env</code> · <code>extends</code> · <code>rules</code> · <code>plugins</code>	<code>.eslintrc</code>

COMMON MISTAKES

- › Using `on:` unquoted in GitHub Actions with YAML 1.1 parsers — some tools parse it as boolean `true`
- › Kubernetes `matchLabels` not matching `template.metadata.labels` — deployment silently manages zero pods
- › Docker Compose `depends_on` only controls start order, not readiness — use healthchecks for true readiness gating
- › Ansible tasks list is a sequence — forgetting the leading `-` on each task causes a parse or runtime error

TIPS

- › Use `kubectl apply --dry-run=client -f manifest.yaml` to validate Kubernetes YAML without deploying
- › Use `docker compose config` to validate and print the fully-merged Compose file before running
- › Use `ansible-playbook --syntax-check playbook.yaml` to catch YAML and Ansible structural errors early

SUMMARY

- › GitHub Actions: `on` · `jobs` · `steps` · `uses` · `run`
- › Docker Compose: `services` · `image` · `ports` · `volumes`
- › Kubernetes: `apiVersion` · `kind` · `metadata` · `spec`
- › Ansible: `hosts` · `become` · `tasks` (sequence of maps)
- › Each platform has a strict schema — validate before applying

PARSING & VALIDATION

Every language has at least one YAML library, but they differ in **YAML version support**, **security defaults**, and **round-trip fidelity**. PyYAML is the most common Python library but defaults to YAML 1.1 and has unsafe loaders. `ruamel.yaml` provides true YAML 1.2 and preserves comments on round-trip. In JavaScript, `js-yaml` is the de facto standard. Always use the **safe** loader/parser variant when loading untrusted input — unsafe loaders can execute arbitrary code via custom tags.

SAFETY & BEST PRACTICES

- Never use `yaml.load()` in Python — always use `yaml.safe_load()`
- `yaml.safe_load()` only resolves standard YAML types; no custom Python objects
- In Node.js, `js-yaml` uses safe schema by default
- Run `yamllint` in CI to catch syntax and style issues before merge
- `yamllint -d relaxed file.yaml` — relaxed profile allows more flexible formatting
- VS Code: install `YAML` extension by Red Hat for schema validation and autocompletion
- JSON Schema can validate YAML — many tools support `$schema` in YAML files

NOTES

- `yaml.safe_load()` — only use `yaml.load()` with `Loader=yaml.SafeLoader`; bare `load()` can execute code
- `yaml.safe_load_all()` — returns a generator of documents from a multi-document file
- `default_flow_style=False` — forces block style output; `True` produces compact inline YAML
- `ruamel.yaml` — preserves comments, quote styles, and key ordering on round-trip writes
- `yaml.load(f)` — ruamel's load is safe by default when using the `YAML()` constructor with no type argument
- For JavaScript: `const yaml = require('js-yaml'); yaml.load(str)` — same pattern, safe by default
- For Go: annotate structs with `yaml:"field_name"` tags then call `yaml.Unmarshal(data, &cfg)`

READING & WRITING YAML IN CODE

```
# — Python: PyYAML —————
import yaml

# Read YAML file safely
with open("config.yaml") as f:
    config = yaml.safe_load(f)    # returns dict
    # config["app"]["port"] → 8080

# Read YAML string
data = yaml.safe_load("name: Alice\nage: 30")
# → {"name": "Alice", "age": 30}

# Write YAML
output = yaml.dump({"key": "value", "num": 42})
# → "key: value\nnum: 42\n"

# Pretty-print with options
yaml.dump(data, default_flow_style=False, indent=2)

# Multiple documents
docs = list(yaml.safe_load_all(open("multi.yaml")))

# — Python: ruamel.yaml (YAML 1.2 + comment-preserving) ———
from ruamel.yaml import YAML
yaml = YAML()
yaml.preserve_quotes = True

with open("config.yaml") as f:
    data = yaml.load(f)          # preserves comments!

data["version"] = "2.0"
with open("config.yaml", "w") as f:
    yaml.dump(data, f)          # comments stay intact
```

VALIDATION & CLI TOOLS

TOOL	INSTALL	USAGE
<code>yamllint</code>	<code>pip install yamllint</code>	<code>yamllint file.yaml</code>
<code>yq</code>	<code>brew install yq</code>	<code>yq 'key' file.yaml</code>
<code>python -c</code>	built-in	<code>python -c "import yaml,sys; yaml.safe_load(sys.stdin)"</code>
<code>js-yaml CLI</code>	<code>npm i -g js-yaml</code>	<code>js-yaml file.yaml</code>
<code>kubectl validate</code>	<code>kubectl</code>	<code>kubectl apply --dry-run=client -f f.yaml</code>
<code>cfn-lint</code>	<code>pip install cfn-lint</code>	<code>cfn-lint template.yaml (CloudFormation)</code>

LIBRARY QUICK REFERENCE

LANGUAGE	LIBRARY	READ	WRITE
Python	PyYAML	<code>safe_load()</code>	<code>dump()</code>
Python	<code>ruamel.yaml</code>	<code>load()</code>	<code>dump()</code>
JavaScript	<code>js-yaml</code>	<code>load()</code>	<code>dump()</code>
Go	<code>gopkg.in/yaml.v3</code>	<code>Unmarshal()</code>	<code>Marshal()</code>
Java	SnakeYAML	<code>load()</code>	<code>dump()</code>
Ruby	Psych (stdlib)	<code>YAML.load()</code>	<code>to_yaml</code>
Rust	<code>serde_yaml</code>	<code>from_str()</code>	<code>to_string()</code>
.NET	YamlDotNet	<code>Deserializer</code>	<code>Serializer</code>

COMMON MISTAKES

- Using bare `yaml.load()` in Python — always use `yaml.safe_load()` or pass `Loader=yaml.SafeLoader`
- Expecting PyYAML to honor `%YAML 1.2` — it doesn't; use `ruamel.yaml` for 1.2 compliance
- Using `yaml.load_all()` without iterating — it returns a lazy generator, not a list
- Round-tripping with PyYAML loses comments, custom tags, and formatting — use `ruamel.yaml` when preserving these matters

TIPS

- Use `yq` on the command line to query and transform YAML files — `yq '.services.web.image' docker-compose.yaml`
- Add `yamllint` to your pre-commit hooks — catches indentation errors before they reach CI
- Install the VS Code Red Hat YAML extension and point it at JSON Schema files for inline validation and autocomplete

SUMMARY

- `yaml.safe_load()` — always use safe loader in Python
- `ruamel.yaml` — YAML 1.2 + round-trip comment preservation
- `js-yaml` — standard JavaScript YAML parser
- `yamllint` — lint YAML files in CI pipelines
- `yq` — jq-style CLI tool for querying YAML files